

DocMate Laptop Setup and Data Flow

Purpose

This document lists the software, platform components, Python environment, OCR tools, cloud options, and operational data flow required to run DocMate on a laptop.

Recommended Laptop Platform

- Windows 10 or Windows 11, 64-bit
- Minimum 16 GB RAM recommended
- SSD storage recommended
- Stable internet access for first-time package/model downloads and cloud extraction
- Chrome or Microsoft Edge for the Streamlit web interface

Core Software to Install

1. Python 3.10 64-bit
 - Recommended because the project currently uses a `virtualenv310` environment.
 - Avoid changing Python major/minor version unless dependencies are retested.
2. Git for Windows
 - Used for pulling updates and version tracking.
3. Microsoft Visual C++ Redistributable 2015-2022
 - Required by several OCR/image/PDF Python packages.
4. Microsoft ODBC Driver for SQL Server
 - Install ODBC Driver 17 or 18.
 - Current config references ODBC Driver 17 for SQL Server.
5. Tesseract OCR for Windows
 - Recommended install path:
`C:\Program Files\Tesseract-OCR\tesseract.exe`
 - Configure this path inside DocMate Admin settings if needed.
6. Poppler for Windows
 - Optional but useful for PDF processing tools.
 - Add Poppler bin folder to the Windows PATH.
 - Example:
`C:\poppler\Library\bin`
7. NSSM
 - Optional.
 - Used only if the background worker should run as a Windows service.

Project Folder

Recommended location:

```
C:\OCRMATE
```

Important folders:

```
C:\OCRMATE\DocMate_DATA  
C:\OCRMATE\virtualenv310
```

`DocMate_DATA` stores logs, debug OCR output, settings, local template data, and generated artifacts.

Python Virtual Environment Setup

Open PowerShell:

```
cd C:\OCRMATE
py -3.10 -m venv virtualenv310
.\virtualenv310\Scripts\activate
python -m pip install --upgrade pip setuptools wheel
pip install -r requirements.txt
```

Recommended additional packages for Gemini and DocTR:

```
pip install google-genai google-generativeai "python-doctr[torch]"
```

Python Packages Used by DocMate

Main packages from requirements.txt:

- streamlit - web app user interface
- pandas, numpy, scipy - data processing
- pillow, opencv-python - image handling
- pymupdf, pypdf, pdfplumber, pdfminer.six - PDF extraction
- pytesseract - Tesseract OCR bridge
- easyocr - local OCR option
- python-doctr - local DocTR OCR
- pyodbc - SQL Server connection
- cryptography, pyyaml, python-dotenv - settings, secrets, and configuration
- google-cloud-documentai - Google Document AI integration
- genai / google-genai - Gemini integration
- reportlab - PDF/report output support

Cloud/API Configuration

Cloud extraction is optional but recommended as a fallback and teacher path.

Gemini

Required when using Cloud Gemini extraction:

- Gemini API key
- Internet access to Google APIs
- Gemini model configured in DocMate Admin settings

Gemini is used for:

- Better extraction on difficult invoices
- Fallback when local OCR confidence is weak
- Template training teacher output
- Multi-language invoices where local OCR/template extraction is incomplete

Google Document AI

Optional structured extraction tier.

Required only if the customer uses Document AI:

- Google Cloud project
- Document AI processor ID
- Location

- Service account credentials
- Internet access to Google Cloud

Local OCR Engines

DocMate can use multiple local OCR engines:

- PDF text layer extraction
- Tesseract OCR
- DocTR OCR
- EasyOCR

Recommended local-first path:

```
PDF text layer -> DocTR -> template rules -> Gemini fallback if confidence is weak
```

Tesseract is useful as a fallback, but DocTR is preferred for supplier-template training/local OCR workflows.

Running the Streamlit App

For v1.16:

```
cd C:\OCRMATE
.\virtualenv310\Scripts\activate
streamlit run docmate_app_v1.16.001_r01_customer.py
```

For v1.17:

```
cd C:\OCRMATE
.\virtualenv310\Scripts\activate
streamlit run docmate_app_v1.17.001_r01_customer.py
```

Streamlit will open a local browser page, usually:

```
http://localhost:8501
```

Optional Background Worker

If the site uses automated upload processing:

- Configure upload/watch folders.
- Ensure the worker uses the same app version as the Streamlit app.
- Restart the worker after code changes.
- If using NSSM, register the worker as a Windows service.

The worker must have access to:

- The same Python virtual environment
- The same settings/API keys
- The same SQL Server connection
- The same upload folders

SQL Server / Database Requirements

Install:

- SQL Server, Azure SQL, or access to the existing central SQL Server
- Microsoft ODBC Driver 17 or 18

DocMate saves:

- Invoice header records
- Invoice line records
- Supplier templates

- Customer templates
- Training metadata
- Audit/log details where configured

If SQL Server is unavailable, local app data may still exist in DocMate_DATA, but production save workflows should use SQL Server.

Supplier Template Learning

Supplier templates store smart metadata, not a full AI model.

Templates can learn:

- Supplier aliases
- Header aliases
- Line aliases
- VAT mappings
- Currency
- Locale profile
- Document type
- Layout anchors
- Preferred local OCR engine
- Template rules
- Line parser hints

Gemini can be used as the teacher. After reviewing a good Gemini result, train/save the supplier template so later invoices can use local DocTR plus the saved template metadata.

Data Flow

1. Invoice enters DocMate
 - User uploads PDF/image in Streamlit, or
 - Background worker picks up file from a watched folder.
2. File preparation
 - Detect file type.
 - Read PDF text layer if available.
 - Render pages/images if OCR is needed.
3. Template matching
 - Match supplier/customer using saved template keywords and aliases.
 - Load supplier template config if found.
4. Extraction
 - Use text layer/local OCR where possible.
 - Apply supplier parser and template rules.
 - Use Gemini fallback if local/template confidence is weak or cloud is selected.
5. Normalisation and repair
 - Clean header fields.
 - Normalise dates, totals, VAT, currency, and line values.
 - Apply promo/free line rules.
 - Reconcile header totals against line totals.
6. Review
 - User reviews header and line items in Streamlit.

- User can edit values before saving.
7. Save
 - Header and lines are saved to SQL Server/local storage.
 - Logs and debug artifacts are written to DocMate_DATA.
 8. Train template
 - User trains header-only or full supplier template.
 - Corrected values and Gemini output can be saved as template metadata.
 - Future invoices from the same supplier use this metadata.
 9. Future processing
 - Auto mode tries saved template/local OCR first.
 - Gemini remains available as fallback when confidence is weak.

Operational Notes

- Reprocess an invoice after code/template changes; old Streamlit session rows may still show stale data.
- Restart the worker after app code changes.
- Keep API keys secure and avoid sharing local credential files.
- Back up DocMate_DATA and SQL Server data regularly.
- For new suppliers, one good invoice can improve header learning, but line extraction usually improves with multiple trained examples.

Quick Verification Checklist

- Python 3.10 opens successfully.
- `.\virtualenv310\Scripts\activate` works.
- `pip install -r requirements.txt` completes.
- Streamlit app starts.
- Tesseract path is valid in Admin settings.
- DocTR can run at least once and download/load models.
- Gemini API key is configured if cloud fallback is required.
- SQL Server connection succeeds.
- Test invoice can be uploaded, reviewed, saved, and trained.
- Worker, if used, processes a test file after restart.